

HealthSense Allocator

Otimização Evolutiva da Carteira de Customer Success

Trabalho	Parte 2 — IA Evolutiva e Computação Bioinspirada · Opção 2 (Protótipo)
Disciplina	Inteligência Artificial e Computacional (0700M8) — CESUPA
Professor	Daniel Leal Souza · Semestre 2026/1
Equipe	[Integrante 1] · [Integrante 2] · [Integrante 3] · [Integrante 4] · [Integrante 5]
Turma	[CC5MA / CC5NA]
Repositório	https://github.com//healthsense

Resumo

Este trabalho apresenta o **HealthSense Allocator**, um protótipo de apoio à decisão que otimiza a alocação da capacidade de um time de Customer Success em SaaS B2B. Dada uma carteira de clientes, um time de CSMs com orçamentos de horas e especialidades, e um catálogo de *playbooks*, o sistema decide **qual CSM atende qual cliente com qual playbook** para maximizar a receita esperada retida/expandida. O problema é formulado como um **Problema de Atribuição Generalizada (GAP)** e resolvido por um **Algoritmo Genético** cuja função de aptidão reutiliza o sistema fuzzy da Parte 1 para prever o ganho de saúde (ΔCHS) de cada intervenção. O AG supera consistentemente uma heurística gulosa (+2,1% em média, 5 sementes) e a busca aleatória (+55%). Compara-se ainda o AG com uma versão Memética e mostra-se, com a contagem correta de avaliações, que o ganho aparente do memético por geração é um artefato — no eixo justo o AG puro domina. A integração Fuzzy–Evolutiva é decisiva: substituí-la por um modelo linear degrada o valor real da alocação em ~64%.

Palavras-chave: algoritmo genético, GAP, computação evolutiva, lógica fuzzy, Customer Success.

1. Problema e público-alvo

Empresas de SaaS B2B vivem de receita recorrente, e a função de Customer Success (CS) existe para reter e expandir a base. O recurso escasso dessas equipes é o **tempo**: cada CSM gerencia muitas contas e tem um orçamento limitado de horas por ciclo. A cada ciclo é preciso decidir em quais contas investir e com qual ação (playbook).

Público-alvo: líderes de Customer Success (planejamento de capacidade), CS Operations (desenho de playbooks e roteamento) e os próprios CSMs, que recebem o plano de ação priorizado. **Decisão apoiada:** para cada cliente, o par (CSM, playbook) ou nenhuma ação, respeitando o orçamento de horas de cada CSM e maximizando a receita esperada.

2. Por que Computação Evolutiva

- **Espaço combinatório enorme:** 40 clientes × 26 opções cada ≈ 10¹⁰ alocações possíveis.
- **NP-difícil:** é um GAP — múltiplas restrições de capacidade (uma por CSM) e a eficácia depende do trio (cliente, CSM, playbook) via especialidades, acoplando as decisões.

- **Objetivo não-linear e não-diferenciável:** o ΔCHS vem de um sistema fuzzy (operadores mín/máx, base de regras), o que impede programação linear.

Metaheurísticas como o Algoritmo Genético lidam naturalmente com objetivo caixa-preta e espaço combinatório, sendo adequadas a este problema.

3. Formulação da otimização

Variáveis de decisão: vetor de inteiros de tamanho N ; para o cliente i , o gene codifica o par (CSM, playbook) ou “nenhuma ação”. **Função objetivo (aptidão):** maximizar $\sum \text{MRR}_i \times (\Delta\text{CHS}_i/100)$ – penalidade por horas excedidas, onde ΔCHS é o ganho de saúde previsto pelo motor fuzzy. **Restrições:** a soma de horas atribuídas a cada CSM $\leq$ seu orçamento. A factibilidade é garantida por um **operador de reparo** que remove as ações de pior densidade (valor/hora) dos CSMs acima do orçamento.

4. O motor evolutivo

Componente	Escolha	Justificativa
Representação	Vetor de inteiros	Natural para problema de atribuição (uma posição por cliente)
Seleção	Torneio ($k=3$)	Pressão seletiva controlável, robusto à escala de aptidão
Crossover	Uniforme ($p=0,9$)	Cada cliente é independente; mistura bem boas atribuições
Mutação	Por gene ($p\approx 0,03$)	Diversidade local
Elitismo	2 melhores	Não perde a melhor solução
Factibilidade	Operador de reparo	Mantém a população viável (decisivo p/ superar o guloso)
Parada	Gerações / estagnação	Eficiência (pop. 80, até 150 gerações)

Tabela 1 — Componentes do Algoritmo Genético e justificativas.

A versão **Memética** (ampliação) adiciona busca local (hill-climbing) refinando os melhores indivíduos a cada geração; as avaliações da busca local são contabilizadas para uma comparação justa.

5. Implementação

Implementado em Python. O motor fuzzy da Parte 1 (src/fuzzy/) é reutilizado como função de aptidão (**integração Fuzzy–Evolutiva**): para cada ação candidata, aplica-se o efeito do playbook ao estado do cliente e consulta-se o fuzzy para o novo CHS; o ΔCHS vira o valor. O motor evolutivo está em src/evolutionary/ (problema GAP, AG, memético, baselines) e o produto é um dashboard Streamlit (app/pages/2_Allocator.py) que gera e exporta o plano de ação.

6. Experimentos e resultados

6.1. Comparação com métodos simples (5 sementes)

Método	Receita média (R\$)	Observação
Sem otimização	0	Nenhuma ação — piso
Busca aleatória	6.255	Com reparo; muito abaixo do guloso
Guloso (densidade)	9.517	Heurística forte de referência
Algoritmo Genético	9.718	+2,1% vs guloso · +55% vs aleatório (desvio ~42)

Tabela 2 — AG vs. métodos simples. A aleatória com reparo bem abaixo do guloso prova que o ganho vem da busca evolutiva, não do reparo.

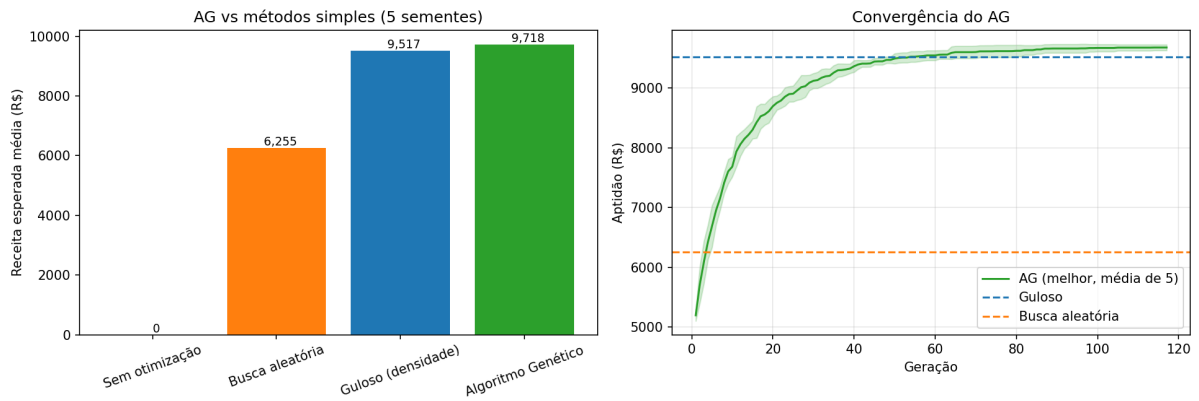


Figura 1 — Receita média por método e convergência do AG (5 sementes).

6.2. O plano de ação gerado (exemplo real)

Saída concreta do AG para a carteira padrão: o sistema concentra horas em contas Em Risco e de Fronteira de alto MRR (onde o ΔCHS é maior) e roteia cada playbook ao CSM da especialidade adequada.

Cliente	CHS	Ação (CSM → Playbook)	ΔCHS	Horas	Valor
Em risco-014	34,7	Bruno (Retenção) → Executive Business Review	+15,3	8h	R\$ 837
Fronteira-017	43,2	Ana (Onboarding) → Onboarding Refresher	+26,3	4h	R\$ 712
Em risco-008	31,8	Elena (Adoção) → Adoption Coaching	+18,2	5h	R\$ 687
Em risco-012	39,0	Carla (Retenção) → Executive Business Review	+11,0	8h	R\$ 596
Em risco-010	36,7	Elena (Adoção) → Onboarding Refresher	+12,2	4h	R\$ 516

Tabela 3 — Trecho do plano de ação gerado pelo AG (R\$ 9.747 totais, 26/40 clientes atendidos, 121 gerações).

6.3. Ampliação obrigatória: AG vs Memético

Método	Fitness médio	Avaliações	Tempo
Algoritmo Genético	9.718	10.298	1,0 s
Memético (AG + busca local)	9.672	41.802 (4,1×)	3,0 s

Tabela 4 — AG vs Memético. No eixo de gerações o memético parece convergir antes; no eixo de avaliações (o justo) o AG puro domina.

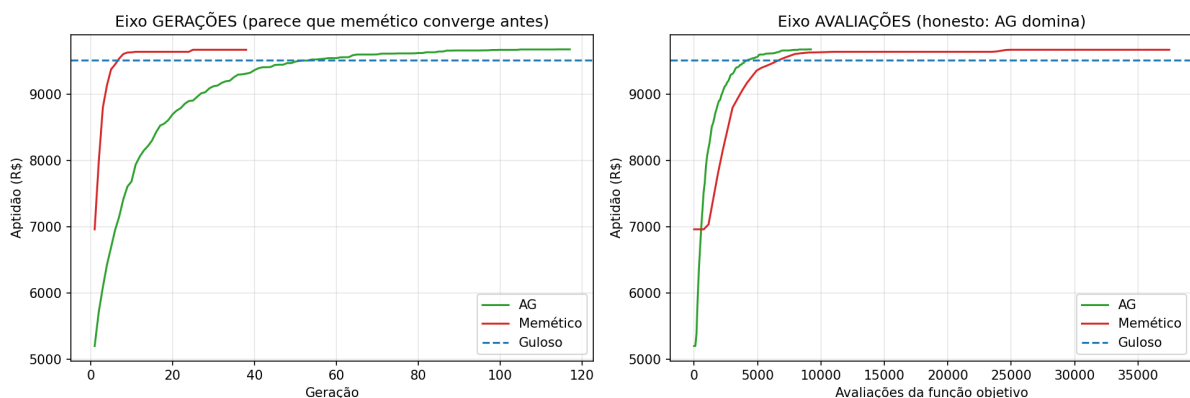


Figura 2 — Convergência por geração (esq., enganoso) vs. por avaliação da função objetivo (dir., justo).

Ampliação cumprida: duas abordagens distintas foram implementadas, comparadas com métricas e gráficos e discutidas com rigor. O enunciado não exige que a versão híbrida vença — exige a comparação e a análise. A conclusão honesta é que, para este GAP, o crossover uniforme já explora bem a estrutura e a busca local de gene único não compensa seu custo.

6.4. Sensibilidade ao regime de capacidade

A vantagem do AG sobre o guloso depende da capacidade: menor sob escassez extrema (o guloso é quase-ótimo) e crescente com a folga (até +7%), onde a miopia do guloso custa caro.

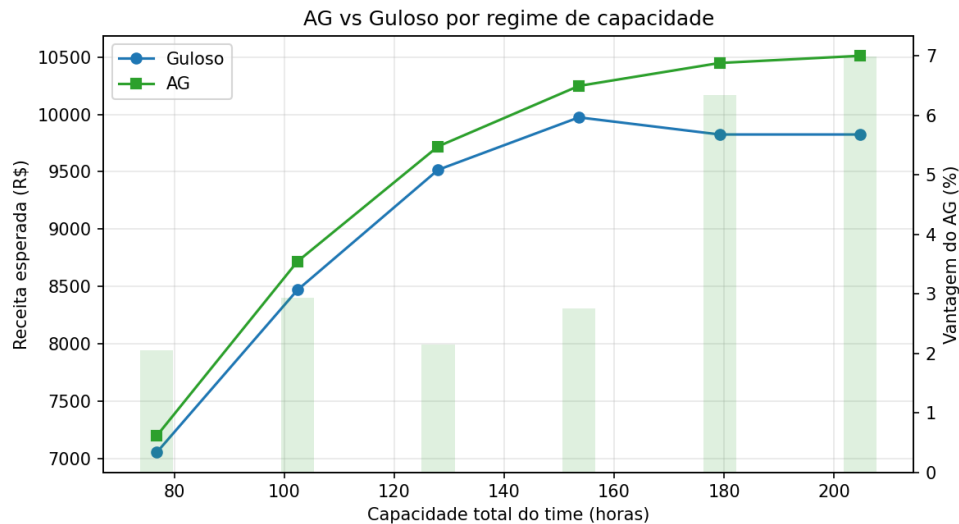


Figura 3 — AG vs guloso por regime de capacidade.

6.5. Extra — impacto da integração Fuzzy–Evolutiva

O *surrogate* linear (que ignora a saturação do CHS) é a alternativa plausível de uma equipe sem o modelo fuzzy. Ele desperdiça horas em contas já saudáveis ($\Delta\text{CHS} \approx 0$). Substituindo a aptidão fuzzy por ele, **29 de 40** alocações mudam e, avaliado no objetivo verdadeiro, o valor cai de **9.747 para 3.484**. Ou seja: o fuzzy não é decorativo — é o que torna a alocação boa.

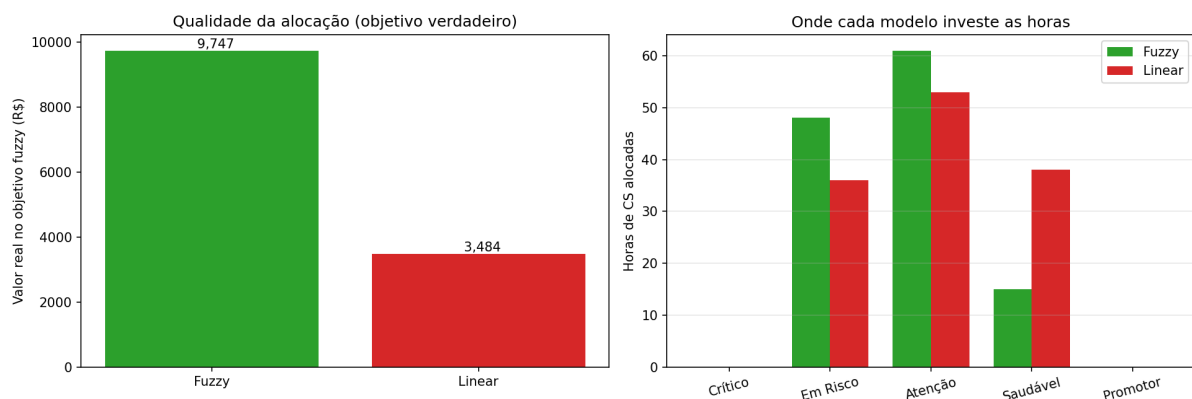


Figura 4 — Fuzzy vs. Linear: valor real no objetivo e onde cada modelo investe as horas.

7. Discussão e limitações

- **Dados sintéticos:** portfólio calibrado por domínio; validação com dados reais é trabalho futuro.
- **Busca local simples:** movimentos de gene único; operadores de troca poderiam mudar a conclusão da Seção 6.3.
- **Uma ação por conta por ciclo:** simplificação do modelo de capacidade.

- **Apoio à decisão:** o plano é uma recomendação; o gestor de CS mantém o controle.

8. Conclusão e trabalhos futuros

O HealthSense Allocator demonstra uma aplicação coerente de Computação Evolutiva a um problema real de Customer Success, com a lógica fuzzy como núcleo de avaliação. O AG entrega alocações melhores que heurísticas simples, de forma estável e reproduzível, num produto demonstrável. Trabalhos futuros: operadores de troca na busca local, múltiplas ações por conta, otimização multiobjetivo (receita × esforço × risco) e integração com CRM real.

9. Declaração de uso de IA

Declarado em detalhe em docs/declaracao_uso_ia.md. Ferramentas de IA apoiaram o esboço de código, a formulação e a revisão textual; todo o material foi revisado, executado e validado pela equipe, inclusive a correção metodológica da comparação AG vs Memético.

10. Referências

- 1 HOLLAND, J. H. *Adaptation in Natural and Artificial Systems*. University of Michigan Press, 1975.
- 2 GOLDBERG, D. E. *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley, 1989.
- 3 MOSCATO, P. On evolution, search, optimization, genetic algorithms and martial arts: towards memetic algorithms. *Caltech Concurrent Computation Program*, Report 826, 1989.
- 4 ZADEH, L. A. Fuzzy sets. *Information and Control*, v. 8, n. 3, 1965.
- 5 MAMDANI, E. H.; ASSILIAN, S. An experiment in linguistic synthesis with a fuzzy logic controller. *Int. J. Man-Machine Studies*, v. 7, n. 1, 1975.
- 6 Optimizing Customer Retention in the Telecom Industry: A Fuzzy-Based Churn Modeling with Usage Data. *Electronics* (MDPI), 13(3), 469, 2024. <https://www.mdpi.com/2079-9292/13/3/469>
- 7 Fuzzy Logic and Decision Making Applied to Customer Service Optimization. *Axioms* (MDPI), 12(5), 448, 2023. <https://www.mdpi.com/2075-1680/12/5/448>
- 8 scikit-fuzzy — Fuzzy Logic Toolkit for SciPy. <https://pythonhosted.org/scikit-fuzzy/>
- 9 Streamlit — The fastest way to build data apps. <https://docs.streamlit.io/>